

INSTITUTO TECNOLÓGICO SUPERIOR DE SAN PEDRO DE LAS COLONIAS



INGENIERÍA EN SISTEMAS COMPUTACIONALES

Fundamentos de Ingeniería de Software

Grupo: 5A

U1-E02 — Análisis individual de problemáticas de Ingeniería de Software

Caso C01 — Sistema de citas de una clínica

PROFESORA:

Ruth Aivi Chávez Rodríguez

ALUMNA:

Bermejo Rodríguez, Ana Cristina

San Pedro, Coahuila de Zaragoza, a 29 de Septiembre de 2026

Presentación del caso y consecuencias

El caso C01 presenta una clínica pequeña que utiliza un sistema web para registrar citas de pacientes con médicos, fechas y horarios. Sin embargo, su funcionamiento básico no garantiza que sea confiable. Existen problemas que afectan tanto al uso diario como al mantenimiento del software.

- Para las recepcionistas, las citas duplicadas y los cambios no registrados generan confusión y trabajo adicional.
- Para el equipo de desarrollo, atender problemas urgentes directamente en producción aumenta el riesgo de introducir errores y dificulta recuperar una versión estable.
- Para la clínica, estas situaciones significan tiempo perdido, discusiones entre áreas y esfuerzo adicional para corregir información.

Por ello, el problema no se limita a la pantalla, sino a la forma en que se desarrolla, prueba y mantiene el sistema.

Cinco problemas identificados

1. Reserva simultánea del mismo horario

Causa: Dos recepcionistas pueden reservar el mismo horario casi al mismo tiempo y ambas operaciones son confirmadas.

Consecuencia: Se pueden generar citas duplicadas y conflictos entre pacientes, médicos y personal administrativo.

Práctica de Ingeniería de Software: Se debe controlar la concurrencia y establecer mecanismos que mantengan la consistencia de los datos. Microsoft explica que cuando varias operaciones modifican los mismos datos al mismo tiempo pueden producirse inconsistencias y que existen mecanismos para manejar estos conflictos (Microsoft, s. f.).

Evidencia: El caso indica que “dos recepcionistas pueden reservar el mismo horario casi al mismo tiempo”.

2. Falta de trazabilidad de los cambios

Causa: No existe un registro claro de quién modificó o canceló una cita y algunos cambios se comunican por mensajería sin actualizar siempre el sistema.

Consecuencia: Cuando aparece una inconsistencia, es difícil saber qué ocurrió y cuál información debe considerarse válida.

Práctica de Ingeniería de Software: Se deben implementar registros de auditoría que permitan identificar las actividades realizadas en el sistema. NIST define un registro de auditoría como evidencia documental de eventos específicos y como un registro cronológico de las actividades del sistema (National Institute of Standards and Technology, s. f.).

Evidencia: El caso señala que no existe un registro claro de quién modificó o canceló una cita.

3. Pruebas insuficientes

Causa: Las pruebas solo comprueban que la pantalla abra y que sea posible guardar una cita.

Consecuencia: Situaciones como reservas simultáneas o modificaciones pueden llegar a producción sin haber sido detectadas.

Práctica de Ingeniería de Software: Se deben ampliar las pruebas para considerar diferentes situaciones y riesgos del sistema. El programa de estudios de ISTQB contempla actividades relacionadas con el proceso de pruebas y el análisis y diseño de pruebas durante el desarrollo de software (International Software Testing Qualifications Board, 2018).

Evidencia: El caso establece que las pruebas se limitan a abrir la pantalla y guardar una cita.

4. Cambios directos en producción

Causa: El equipo publica cambios directamente en producción cuando la clínica reporta un problema urgente, sin un proceso mencionado de revisión y validación previo.

Consecuencia: Una corrección puede introducir otro error y, sin una versión estable identificada, resulta más difícil regresar a un estado anterior.

Práctica de Ingeniería de Software: Se debe establecer un proceso de control de cambios y versiones, acompañado de pruebas antes de liberar modificaciones. El material de ISTQB incluye la gestión de configuración dentro de las actividades relacionadas con el desarrollo y las pruebas de software (International Software Testing Qualifications Board, 2018).

Evidencia: El caso indica que los cambios se publican directamente en producción ante problemas urgentes.

5. Falta de respaldo y recuperación comprobada

Causa: No existe un procedimiento acordado para respaldar información, recuperar una versión anterior ni comprobar que la recuperación funcione.

Consecuencia: Ante una falla, la clínica podría tener dificultades para recuperar información confiable y continuar sus actividades.

Práctica de Ingeniería de Software: Se debe establecer un procedimiento de respaldo y recuperación que permita restablecer las operaciones ante una interrupción. La guía de planificación de contingencia de NIST aborda estrategias y procedimientos para la recuperación de sistemas y operaciones.

Evidencia: El caso señala que no existe un procedimiento acordado para respaldar información ni recuperar una versión anterior.

Tres acciones prioritarias

1. Corregir el problema de citas duplicadas

Justificación: Debe atenderse primero porque afecta directamente la operación de la clínica y puede producir información incorrecta desde el momento en que se registra una cita.

Costo o limitación: El equipo tendrá que modificar la lógica de reserva y dedicar tiempo a realizar pruebas con operaciones simultáneas antes de liberar el cambio. La principal limitación sería no poder publicar la corrección inmediatamente.

Comprobación: Se realizarán pruebas con dos usuarios intentando reservar el mismo horario al mismo tiempo. La mejora se comprobará si el sistema impide la duplicidad y conserva una sola reserva válida.

2. Establecer control de cambios, pruebas y versiones

Justificación: Una vez atendido el problema inmediato, es necesario evitar que nuevas correcciones generen errores adicionales.

Costo o limitación: El equipo necesitará tiempo adicional para revisar, probar y documentar cada cambio, por lo que las correcciones urgentes podrían tardar más en llegar a producción.

Comprobación: Cada cambio deberá tener una versión identificada, evidencia de pruebas y una versión anterior disponible para recuperación.

3. Implementar trazabilidad y respaldo con recuperación comprobada

Justificación: Esta acción permitirá saber qué ocurrió con una cita y recuperar información ante una falla.

Costo o limitación: Requiere espacio para respaldos y tiempo del personal técnico para realizar restauraciones de prueba. Además, será necesario definir qué acciones deben registrarse.

Comprobación: Se hará una restauración controlada de un respaldo y se verificará que la información pueda recuperarse. También se comprobará que las modificaciones y cancelaciones registren responsable y fecha.

Reflexión personal

Este caso me hace ver que desarrollar software no consiste solamente en conseguir que un programa funcione. También es necesario considerar qué ocurre cuando varias personas lo utilizan al mismo tiempo, cuando se modifica información o cuando aparece una falla.

Si desarrollara un sistema como este, daría más importancia a las pruebas antes de publicar cambios y evitaría modificar directamente producción sin una revisión previa. También consideraría desde el

inicio la trazabilidad, los respaldos y la recuperación, porque son aspectos que pueden parecer secundarios mientras todo funciona, pero se vuelven importantes cuando aparece un problema.

Finalmente, considero que una buena práctica es identificar los riesgos antes de que se conviertan en incidentes. De esta manera, el desarrollo no se enfoca solamente en entregar funciones, sino también en mantener un sistema confiable y controlable.

Referencias

International Software Testing Qualifications Board. (2018). *ISTQB certified tester foundation level syllabus: Version 3.1.1*. <https://istqb.org/certifications/certified-tester-foundation-level-ctfl-v3-1-retired/>

Microsoft. (s. f.). *Handling concurrency conflicts*. Microsoft Learn. <https://learn.microsoft.com/en-us/ef/core/saving/concurrency>

National Institute of Standards and Technology. (s. f.). *Audit log*. Computer Security Resource Center. https://csrc.nist.gov/glossary/term/audit_log

Swanson, M., Bowen, P., Phillips, A., Gallup, D., & Lynes, D. (2010). *Contingency planning guide for federal information systems* (NIST Special Publication 800-34 Rev. 1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-34r1>

Declaración de uso de inteligencia artificial

Uso de IA: Utilicé ChatGPT para apoyar la organización de ideas y revisar la redacción en cuestiones ortográficas y de usar un lenguaje formal. Verifiqué el contenido mediante el caso proporcionado por la docente y las fuentes consultadas, y redacté el análisis final a partir de esa información.